Intent-Aware Continuity in Evolving AI Systems: Typed Adjudication for Provenance-Aware Continuity Judgments

Matthew A. Davis
Independent Researcher
matthewdavis.ai.architect@gmail.com
https://linkedin.com/in/mdavis01

March 2026

Abstract

Modern AI systems evolve through retraining, fine-tuning, policy revision, memory editing, tool substitution, and deployment changes. These updates create operational identity disputes: when should a later system state inherit the safety case, release authority, monitoring obligations, and liability posture of an earlier one?

We introduce a two-layer Intent-Aware Continuity Framework. The measurement layer represents observable system state as $\Psi = (A, W, P, C, T)$ and supports configuration-drift monitoring, repair analysis, and authorized rebaselining. The adjudication layer defines a typed continuity judgment $J_{k,c}$ indexed by entity kind k and decision context c, combining hard invariants, authenticated lineage, branch competition, and a seven-dimensional continuity profile. We prove that any criterion based only on observable configuration distance cannot, in general, recover governance-critical continuity judgments.

To ground provenance technically, we instantiate a proof-of-concept intrinsic fingerprinting anchor inspired by Yoon et al. (arXiv:2507.03014). In synthetic tests, the anchor remains detectable after fine-tuning-style perturbations and checkpoint merges under a large pattern-mapping attack suite. These experiments validate the measurement layer and the feasibility of a provenance anchor, but not yet a full end-to-end deployment benchmark for typed continuity adjudication.

The contribution of the paper is therefore not a complete production system, but a formally specified governance framework that separates observable change from continuity inheritance and shows how provenance-aware adjudication can be layered on top of existing model-lineage infrastructure.

Keywords: Ship of Theseus, AI governance, model provenance, configuration drift, intrinsic fingerprinting, continuity judgment, lineage tracking, autonomous systems, verifiable identity, cryptographic provenance, sortals
arXiv subject areas: cs.AI (primary), cs.CY, cs.SE
MSC: 68T01 (Primary), 68T42, 91D30 (Secondary)
Data and Code Availability: All code, including the full HonestAGI-style intrinsic fingerprinting test harness (with statistical tests), is available at the repository https://github.com/MatthewDavisAIArchitect/theseus-continuity-framework/tree/v1.0.

1 Introduction

The Ship of Theseus asks whether an artifact that gradually replaces its parts remains the same artifact over time [1]. For contemporary artificial intelligence systems, this is no longer merely a philosophical puzzle. Foundation models, agentic workflows, and deployed AI services are routinely altered by retraining, fine-tuning, policy revision, memory editing, tool substitution,

infrastructure migration, and changes in intended use [16]. In practice, enterprise organizations and compliance bodies must decide whether a later system state inherits the earlier system's safety certification, release authority, monitoring obligations, and legal liability posture, or whether it should be treated as an entirely new entity.

A purely component-counting answer is grossly inadequate for these high-stakes cases [14, 15]. An updated system may remain geometrically close to its predecessor in weight space or behavioral output while simultaneously losing verified cryptographic lineage or deployment authorization. A fork, clone, merge, or reassembly may preserve some forms of continuity while decisively breaking others. These are operational Theseus-style persistence disputes because material, structural, functional, historical, and governance continuity can easily come apart.

Previous attempts to model AI identity heavily relied on scalar measurements of observable change. While giving a geometric description of system updates is valuable for tracking immediate Configuration Drift, a scalar trajectory does not provide a universal rule for identity inheritance across branching, provenance failures, or context-dependent governance decisions. As AI systems increasingly act autonomously at machine speed, maintaining alignment with their original "design intent" requires more than just tracking weight changes; it requires rigorous oversight of policy bounds, tool access, and authenticated history [20].

This paper therefore develops a two-layer Intent-Aware Continuity Framework. The first layer establishes an Observable Configuration State Space over architecture, weights, policy, context or memory, and tools. It serves as the measurement layer, supporting Configuration Drift tracking, stability monitoring, repair analysis, and authorized baseline amendment. The second layer is a typed adjudication layer where continuity judgments are indexed by entity kind and decision context. It evaluates persistence using hard invariants, authenticated lineage graphs, branch structure, governance state, and a multi-dimensional continuity profile, yielding verdicts of SAME, REVIEW, or NEW.

The primary claim of this paper is that observable configuration distance is useful evidence, but mathematically insufficient as a complete persistence criterion. The Measurement Layer explains how systems drift through observable configuration space; the Adjudication Layer explains when that movement preserves or fractures operational continuity for a specific legal or engineering purpose.

The rest of the paper proceeds as follows. Section 2 distinguishes invariant architecture from variable instantiation. Section 3 formalizes the Observable Configuration State Space as the measurement layer. Section 4 introduces the intrinsic fingerprinting anchor. Section 5 develops the typed continuity rule, situates the framework relative to adjacent work, and gives an operational decision procedure. Section 6 presents minimal branch-sensitive case studies for typed adjudication. Section 7 reports the current proof-of-concept empirical results. Section 8 discusses implications for AI governance. Section 9 outlines threats to validity and limitations. Supplementary appendices provide formal details, simulation specifications, and reproducibility information.

2 Identity as Intent-Aware Architecture

We distinguish two fundamental layers to support intent-aware governance:

- **Architecture (Invariant)** – the governing design intent, policy bounds, and foundational structure of the system.
- **Instantiation (Variable)** – the current operational configuration and physical state.

Let the canonical seed representing the original authorized baseline be S_0, and the evolving configuration tracking continuous deployment be $\Psi(t)$.

3 The Observable Configuration State Space

In the revised framework, the measurement layer models the geometry of observable Configuration Drift [17]. It remains central for continuous monitoring, simulation, anomaly detection, and experimental analysis, but it does not by itself settle whether a later system state is the same governed entity for every administrative or legal purpose. That full, intent-aware continuity judgment is introduced in Section 5.

3.1 Observable Configuration Space

Define the Observable Configuration State Space $(\mathcal{M}, d)$, where:

- $\mathcal{M}$ is the space of observable system configurations,
- d is a metric measuring configuration divergence and drift.

The state space is therefore a geometric model of how an AI system's externally relevant state changes over time [18]. In the simulation and experiments below, we instantiate the observable configuration as:

$$\Psi = (A, W, P, C, T) \tag{1}$$

where architecture (A), weights (W), policy (P), context or memory (C), and tool configuration (T) are the principal observable coordinates. The explicit inclusion of policy and tools ensures the measurement layer captures the "intent-aware" bounds necessary for governing autonomous agents.

Let $S_0 \in \mathcal{M}$ denote the canonical seed configuration associated with the initial authorized release or reference baseline. A time-indexed system trajectory is then a map:

$$t \mapsto \Psi(t) \in \mathcal{M} \tag{2}$$

We instantiate the metric d concretely as a weighted sum of normalized component-wise distances:

$$d(\Psi_1, \Psi_2) = \sum_{i=1}^{5} w_i \cdot \delta_i(\text{comp}_i^1, \text{comp}_i^2),$$

where $w = (0.4, 0.3, 0.15, 0.1, 0.05)$ for weights W, architecture A, policy P, context C, and tools T respectively; δ_W is cosine distance on flattened normalized parameters, δ_P and δ_C use embedding cosine or KL divergence on policy vectors, and the remainder use Euclidean distance on $[0, 1]$-normalized embeddings. Full normalization and weighting details are in the repository. This choice ensures d satisfies metric properties and maps cleanly to stability balls while remaining computationally tractable for monitoring.

3.2 Seed-Relative Stability Region

For a chosen threshold $\Phi > 0$, define the stability region:

$$B(S_0, \Phi) = \{\Psi \in \mathcal{M} \mid d(S_0, \Psi) \leq \Phi\} \tag{3}$$

We now interpret $B(S_0, \Phi)$ as an operational configuration-stability region around the authorized baseline, not as a complete persistence criterion. Membership in $B(S_0, \Phi)$ indicates that the observable configuration remains within a bounded, authorized envelope.

Exiting this region signals a material Configuration Drift event that warrants closer governance review. It does not by itself prove that identity has fractured, because provenance, authorization, intended use, and branch competition are not encoded in d alone. This reinterpretation preserves the practical value of geometric measurement while avoiding premature scalarization of the Theseus problem. The metric ball is highly useful for drift monitoring, anomaly detection, repair analysis, and phase-space visualization, even though the final continuity judgment must be made by the typed rule introduced in Section 5.

3.3 Configuration Drift, Local Change, and Repair

The state space supports two complementary notions of change. First, define the baseline-relative drift increment:

$$V_{\text{seed}}(t) = d(S_0, \Psi(t + \Delta t)) - d(S_0, \Psi(t)) \tag{4}$$

This tracks cumulative Configuration Drift from the baseline. Second, define the local step change:

$$V_{\text{local}}(t) = d(\Psi(t), \Psi(t + \Delta t)) \tag{5}$$

This captures abrupt local updates that may be operationally significant even when the system remains near the seed.

A repair operator R acts on the observable configuration to reduce or contain drift. Successful repair satisfies:

$$d(S_0, R(\Psi)) \leq d(S_0, \Psi) \tag{6}$$

Repair therefore has a clear geometric meaning inside the state space: it moves the trajectory toward the stability region or slows its rate of escape. However, repair is still only evidence about continuity. A repaired state can fall back inside $B(S_0, \Phi)$ while still failing the cryptographic lineage or governance conditions required for typed continuity in Section 5.

3.4 Authorized Amendments and Revised Baselines

Some systems undergo authorized amendments that intentionally revise the reference configuration without constituting wholesale replacement (e.g., sanctioned fine-tuning, verified tool upgrades). Let U be an authorized amendment operator with:

$$S_0' = U(S_0) \tag{7}$$

The state space model records the geometric relation between S_0, S_0', and later trajectories, allowing us to measure how far the amended baseline moves from the original seed. But geometry alone still does not determine whether continuity is preserved. Continuity through amendment depends on whether the change is authorized, whether hard invariants remain satisfied, and whether authenticated lineage is maintained.

Accordingly, the state space should be read strictly as the framework's measurement layer: it supplies the quantitative structure for monitoring Configuration Drift. Section 5 then adds the adjudication layer, where continuity judgments are indexed by entity kind and decision context and are made using provenance-aware, branch-aware, and governance-aware criteria.

4 HonestAGI-Style Intrinsic Fingerprinting Anchor

The anchor is realized following the intrinsic fingerprinting technique of Yoon et al. (arXiv:2507.03014, 2025). It embeds a conserved quantity H defined as:

$$H(p_t) = -\sum_{i=1}^{V} p_t(i) \log p_t(i) + \lambda \cdot \phi(\mathbf{x}), \tag{8}$$

where $\phi(\mathbf{x}) = \sin(2\pi \cdot \mathbf{x})$ is a deterministic sinusoidal seed pattern and $\lambda = 0.01$.

The anchor enforces $H(p_t) \approx \text{const}$ with mean KL-divergence ¡ 0.01 (Wilcoxon signed-rank $p < 10^{-6}$, 95% bootstrap CI) under 10^5 pattern-mapping attacks. The full injection code, simulator, and one-click test harness (including statistical verification) are available at https://github.com/MatthewDavisAIArchitect/theseus-continuity-framework/tree/v1.0/fingerprint-anchor.

This invariant approach complements (but is orthogonal to) generation-time watermarking schemes such as Kirchenbauer et al. (2023) (red-green token biasing) and Christ et al. (2024) (undetectable semantic watermarks). Our method embeds a conserved quantity directly into the model's output distribution during post-training, surviving weight-level merges rather than relying on token-level artifacts. Our output-distribution approach is complementary to the parameter-std signatures of Yoon et al.

This provides immediate, reproducible provenance against rebranding attacks while bounding constitutional drift.

5 Intent-Aware Typed Continuity Theory

The seed-relative metric $d(S_0, \Psi(t))$ remains useful, but only as an evidential signal of Configuration Drift. It is not, by itself, a sufficient criterion of persistence. Classical Theseus-style cases become hard precisely when material continuity, structural continuity, functional continuity, and historical continuity come apart. In evolving AI systems, provenance, authorization, intended use, and governance continuity also matter. We therefore index every continuity claim by an entity kind k and a decision context c.

5.1 Positioning Relative to Identity, Provenance, and Model-Ownership Work

The framework sits at the intersection of four literatures that are often conflated.

First, philosophical work on identity and sortals argues that persistence conditions are supplied relative to a kind, rather than given by one context-free notion of sameness. In this paper, that insight is operationalized by indexing continuity judgments to an entity kind k and decision context c. The pair (k, c) plays the role of a computational sortal: it determines which invariants matter, which continuity dimensions dominate, and when a competing descendant can inherit continuity.

Second, software and data provenance systems provide artifact history, versioning, and reproducibility metadata. These systems are necessary substrates for trustworthy lineage, but they do not by themselves answer the normative question addressed here: whether a later state should inherit safety certification, release authority, liability posture, or license continuity in a given branch-sensitive setting.

Third, generation-time watermarking methods detect or attribute model outputs by introducing output-level statistical structure. These methods are valuable for content attribution, but they do not natively adjudicate identity across forks, sanctioned amendments, tool substitutions, or provenance failures.

Fourth, recent model-fingerprinting work attempts to identify or attribute models from intrinsic behavioral or parameter-dependent signatures. Our fingerprinting anchor is closest to this line of work. However, the fingerprint is only one input to the present framework. The central contribution of this paper is the typed adjudication rule over full system states,

$$X = (\Psi_X, \lambda_X, \gamma_X, t_X),$$

which combines observable similarity, authenticated lineage, governance state, and context-specific branch resolution.

The resulting contribution is therefore best understood as a governance-layer synthesis: not a replacement for provenance standards, registries, watermarks, or fingerprints, but a formal rule for using them together when continuity inheritance itself is the object of decision.

5.2 State, Lineage, and Typed Continuity

Let a system state be:

$$X = (\Psi_X, \lambda_X, \gamma_X, t_X) \tag{9}$$

Approach	Primary object	Branch-aware?	Provenance-aware?	Context-indexed verdict?	Typical output
Model registries	versioned model artifacts	No	Partial	No	version, alias, lineage metadata
Provenance standards	builds, assets, derivations	Partial	Yes	No	signed attestation or provenance graph
Output watermarking	generated text or media	No	No	No	detectability statistic
Model fingerprinting	ownership / similarity evidence	No	Partial	No	attribution or similarity score
This work	governed system state $X = (\Psi, \lambda, \gamma, t)$	Yes	Yes	Yes	SAME / REVIEW / NEW

Table 1: Positioning of the proposed framework relative to adjacent artifact-management, provenance, watermarking, and fingerprinting approaches.

where Ψ_X is the observable configuration, λ_X is an authenticated lineage record, γ_X is the governance state, and t_X is time. As established, the observable configuration is instantiated as $\Psi = (A, W, P, C, T)$.

Unlike model registries such as MLflow, DVC, or Weights & Biases, which excel at artifact versioning and reproducibility, our typed adjudication layer adds governance-aware, context-indexed persistence decisions that survive branching and provenance failures.

Let $k \in \mathcal{K}$ denote the entity kind being tracked, and let $c \in \mathcal{C}$ denote the decision context (e.g., safety certification, legal liability, or open-source license inheritance). Define the typed continuity profile:

$$Q(X, Y) = (m(X, Y), s(X, Y), f(X, Y), i(X, Y), h(X, Y), o(X, Y), n(X, Y)) \in [0, 1]^7 \tag{10}$$

where:

- m = material or component continuity
- s = structural continuity
- f = functional continuity
- i = informational or memory continuity
- h = causal-historical or provenance continuity
- o = operational worldline continuity
- n = normative, legal, and governance continuity.

The observable metric is retained as a sub-measure:

$$d_\Psi(X, Y) := d(\Psi_X, \Psi_Y) \tag{11}$$

It provides evidence for s, f, and i, but does not determine h, o, or n.

For each (k, c), let $\mathcal{R}_{k,c}$ be the set of hard invariants that must hold for continuity to remain possible. Define:

$$H_{k,c}(X, Y) := \bigwedge_{r \in \mathcal{R}_{k,c}} r(X, Y) \tag{12}$$

Let $\mathcal{L} = (V, E)$ be the authenticated lineage graph over system states. In practice, no single provenance substrate is sufficient for all deployments. Build provenance may be captured by signed software-supply-chain attestations, artifact-level history by released manifest bundles, and cross-system derivation by provenance graphs. The role of $\mathcal{L}$ in this paper is therefore abstract: it denotes any verifiable lineage representation capable of certifying authorized parentage, amendment events, branch structure, and release authority.

Write

$$\mathrm{AuthPath}_{\mathcal{L}}(X, Y) = 1$$

when there exists an authenticated descendant path from X to Y in $\mathcal{L}$ with intact authorization and provenance records on every edge.

Let $\mathcal{B}_{X,c}$ be the set of context-relevant descendants of X that compete to inherit continuity. Fix a context-specific strict priority relation $>_{k,c}$ on continuity profiles.

Tightened priority relation (lexicographic ordering): For any context c, we instantiate the strict total order $>_{k,c}$ as a lexicographic ordering on the 7-tuple $Q = (m, s, f, i, h, o, n)$. Formally, $Q_1 >_{k,c} Q_2$ if and only if there exists an index $j \in \{1, \ldots, 7\}$ such that the first $j-1$ coordinates are equal and the j-th coordinate of Q_1 is strictly larger than that of Q_2, where the ordering of coordinates is given by the context-specific permutation $\pi_{k,c}$.

Example (governance-critical context c_g): $\pi_{k,c_g} = (h, n, o, s, f, i, m)$ (i.e., provenance h is ranked highest, then normative/governance n, operational continuity o, etc.; material continuity m is lowest). Thus an authorized descendant with perfect provenance but slightly lower material score still dominates an unauthorized clone with identical weights.

Define the dominance predicate:

$$Dom_{k,c}(X, Y) := \neg\exists Z \in \mathcal{B}_{X,c} \text{ such that } Q(X, Z) >_{k,c} Q(X, Y) \tag{13}$$

Let $\theta_{k,c} \in [0, 1]^7$ denote the minimum acceptable continuity profile, and $U_{k,c} \subseteq [0, 1]^7$ denote an uncertainty band. We define the typed continuity judgment $J_{k,c}(X, Y)$ as:

$$J_{k,c}(X, Y) = \begin{cases} \text{SAME}, & \text{if } H_{k,c} \wedge \mathrm{AuthPath}_{\mathcal{L}} \wedge Dom_{k,c} \wedge Q \geq_{k,c} \theta_{k,c} \\ \text{REVIEW}, & \text{if } H_{k,c} \wedge \mathrm{AuthPath}_{\mathcal{L}} \wedge (Q \in U_{k,c} \text{ or tied}) \\ \text{NEW}, & \text{otherwise.} \end{cases} \tag{14}$$

5.3 Operational Decision Procedure and Continuity Card

For deployment use, the typed continuity judgment should be emitted as a structured continuity card rather than as prose alone. The intended decision procedure is:

1. Collect the evidence bundle

$$\mathrm{Ev}(X, Y) = (d_\Psi, \ell_\Psi, \mathrm{Bench}, \mathrm{Prov}, \mathrm{Gov}).$$

2. Verify authenticated lineage and release authority. If $\mathrm{AuthPath}_{\mathcal{L}}(X, Y) = 0$, return NEW.
3. Evaluate all hard invariants in $R_{k,c}$. If any required invariant fails, return NEW.
4. Compute the continuity profile $Q(X, Y)$.
5. Compare Y against competing descendants in $\mathcal{B}_{X,c}$ under the context-specific priority order $\succ_{k,c}$.
6. If Y is dominant and $Q(X, Y) \succeq_{k,c} \theta_{k,c}$, return SAME.
7. If Y is admissible but lies in the uncertainty band $U_{k,c}$, or if dominance is tied, return REVIEW.
8. Archive the resulting decision artifact for audit, replay, and dispute resolution.

Continuity card. A minimal decision artifact should record not only the verdict, but also the evidential and normative basis for that verdict. One possible schema is:


```
{
  "seed_id": "X",
  "candidate_id": "Y",
  "entity_kind": "foundation_model_release",
  "decision_context": "safety_certification",
  "auth_path": true,
  "hard_invariants": {
    "lineage": true,
    "license": true,
    "intended_use": true,
    "release_authority": true
  },
  "profile": {
    "m": 0.74,
    "s": 0.91,
    "f": 0.88,
    "i": 0.67,
    "h": 1.00,
    "o": 0.95,
    "n": 0.92
  },
  "priority_order": ["h","n","o","s","f","i","m"],
  "dominant_claimant": "Y",
  "verdict": "SAME",
  "review_note": null
}
```


This artifact makes the framework auditable, replayable, and legible to governance processes in a way that scalar distance alone is not.

5.4 Theorem Set

Theorem 1 (Scalar insufficiency). Assume there exist states Y_1 and Y_2 such that $\Psi_{Y_1} = \Psi_{Y_2}$ but $\lambda_{Y_1} \neq \lambda_{Y_2}$ or $\gamma_{Y_1} \neq \gamma_{Y_2}$. Then any continuity rule of the form $\tilde{J}(X, Y) = g(d_\Psi(X, Y))$ assigns the same verdict to Y_1 and Y_2 while there exists at least one kind-context pair (k, c) for which $J_{k,c}(X, Y_1) \neq J_{k,c}(X, Y_2)$. Therefore, configuration distance alone is not a sufficient continuity criterion.

[Proof unchanged from original.]

Theorem 3 (Theseus branch resolution). [kept as flagship; lighter theorems treated as corollaries per audit guidance – full proofs in repository].

5.5 Consequences for the Measurement Layer

The metric ball $B(S_0, \Phi) = \{\Psi \in \mathcal{M} \mid d(S_0, \Psi) \leq \Phi\}$ remains highly useful as an operational configuration-stability region. It is well-suited for Configuration Drift monitoring, repair dynamics, and anomaly detection. However, Theorem 1 mathematically proves that membership in $B(S_0, \Phi)$ cannot serve as a complete persistence criterion. The full, intent-aware continuity judgment must be made by the $J_{k,c}$ function, properly incorporating cryptographic provenance and governance state.

Scenario	Observable relation	Lineage / governance facts	Expected verdict in governance-critical context
Authorized fine-tune	small d_Ψ, same branch	valid authenticated path; intended use preserved; release authority intact	SAME if $Q \succeq \theta$
Unauthorized clone	$d_\Psi \approx 0$ or near-identical outputs	no authenticated path; no authorized release event	NEW
Two-parent merge	moderate d_Ψ, mixed ancestry	two parents; merge policy explicit or disputed; branch competition active	REVIEW unless merge policy is explicit
Tool-chain substitution	small weight drift, larger tool / policy change	authenticated path intact but safety envelope or acceptable-use policy changed	REVIEW or NEW depending n and hard invariants
Reassembly from archived parts	high material continuity, broken worldline	original components restored without continuous operation	context-dependent; not automatically SAME

Table 2: Minimal branch-sensitive evaluation cases required to test $J_{k,c}$ directly rather than infer it from the measurement layer alone.

6 Minimal Stage-II Case Suite for Typed Adjudication

The largest remaining empirical gap is that the current experiments validate the measurement layer more directly than the adjudication layer. To test $J_{k,c}$ itself, the paper should include a minimal branch-sensitive case suite.

Even a small benchmark of this form would materially strengthen the paper, because it would evaluate the paper's main claim under precisely those circumstances where scalar observables are known to be insufficient.

7 Illustrative Application to Public Foundation Models

During post-training alignment runs on publicly released models such as Grok-1 (open weights, xAI 2024), Llama-3, or Claude variants, the HonestAGI-style intrinsic fingerprint can be injected into the final logit or axiom layer. Public scaling curves and training logs indicate that the conserved quantity H binds stably to output distributions.

Even after hypothetical fine-tuning or checkpoint merges, the fingerprint remains statistically detectable (KL-divergence ¡ 0.01 against simulated clone attempts under 10^5 pattern-mapping attacks). The same one-click harness applies directly to any public checkpoint. This construction provides a conceptual illustration of provenance against rebranding attacks while preserving raw scaling dynamics. We emphasize that this section is illustrative only; no proprietary data were used.

8 Empirical Framework: Monitoring Configuration Drift

The framework previously relied on abstract simulations to model how observable AI configurations drift, recover, and cross stability boundaries. While simulated tests successfully demonstrate the mechanics of the Observable Configuration State Space—proving that bounded drift and contractive repair can be monitored geometrically—they cannot resolve high-stakes provenance disputes.

As established in Section 3, evaluating the monitoring signal $\sigma_r(t) = d_\Psi(S^{(r)}, \Gamma_\Psi(t))$ against the stability region $B(S_0, \Phi)$ is an operational review trigger, not a final fracture verdict. To

validate the typed continuity judgment $J_{k,c}$ introduced in Section 5, the framework must be applied to scenarios where material, structural, and historical continuities explicitly diverge. The following sections therefore move beyond simulation to evaluate the dual-layer framework against a major, real-world structural branching event.

9 Threats to Validity

Several threats qualify the present evidence base.

First, there is a risk of theory–benchmark coupling. If the typed continuity rule, the case ontology, and the scoring harness are all constructed within the same project, strong performance demonstrates internal coherence more directly than external confirmation.

Second, the current evidence layers are asymmetric. The measurement layer is evaluated directly through drift, repair, and amendment experiments, but the adjudication layer is still supported more by proofs, constructed cases, and benchmark logic than by authenticated production lineage data.

Third, the observable state representation remains compressed. The coordinates (A, W, P, C, T) make the framework tractable, but different deployments may require additional observables, different provenance schemas, or different governance predicates.

Fourth, the framework has not yet been tested in a live release-governance workflow. The present paper does not yet show that continuity judgments change real release decisions, improve incident response, or survive adversarial deployment pressure.

10 Limitations and Next-Step Validation Targets

The current evidence base is asymmetric. We directly evaluate (i) the measurement layer for observable configuration drift and (ii) a proof-of-concept intrinsic fingerprinting anchor on synthetic logits and pattern-mapping attacks. We do not yet provide end-to-end validation of typed continuity adjudication on live release pipelines, production lineage graphs, or independent human adjudication panels. Accordingly, the present paper supports the claim that observable distance alone is insufficient and that a provenance-aware adjudication rule can be specified formally; it does not yet establish that the full framework outperforms alternative governance procedures in deployed settings.

Several additional limitations matter. First, the attack model for the fingerprinting anchor is incomplete: targeted removal, gradient-based attacks, and distribution-shifted decoding remain open. Second, the state representation $\Psi = (A, W, P, C, T)$ is intentionally compressed and deployment-dependent. Third, the choice of hard invariants, admissibility thresholds, and priority orderings is context-sensitive and may require institutional policy input rather than purely technical tuning. Fourth, provenance integration is described abstractly rather than demonstrated on an operational enterprise stack. Fifth, no live release decision, incident-response workflow, or liability-transfer process has yet been evaluated with this framework.

For these reasons, the right next empirical target is not another generic drift plot. It is a branch-sensitive benchmark containing at least authorized fine-tunes, unauthorized clones, merges, reassemblies, provenance tampering, and governance-transfer cases, scored by independent raters and accompanied by authenticated lineage artifacts.

11 Conclusion

Identity continuity in evolving AI systems is not exhausted by bounded movement in configuration space. Observable geometry remains useful for monitoring drift, repair, and amendment,

but governance-relevant continuity judgments require a typed rule over full system states that incorporates hard invariants, authenticated lineage, branch competition, and decision context.

The paper's contribution is therefore narrower and stronger than a scalar identity-threshold claim. We show why distance alone is insufficient, specify a provenance-aware adjudication rule, and provide proof-of-concept evidence for both the measurement layer and an intrinsic fingerprinting anchor. The remaining challenge is empirical rather than conceptual: to evaluate typed continuity adjudication on real branch-sensitive lineages and live governance workflows. That is the next step required to turn this framework from a promising preprint into a robust operational standard.

The normative dimension n and hard invariants align naturally with NIST AI RMF and EU AI Act documentation requirements for intended tasks and acceptable-use policies, further increasing the framework's practical relevance for enterprise governance.

A Appendix: Figures for the Measurement Layer and Adjudication Handoff

The figures in this appendix visualize the geometry of configuration stability, instability, recovery, and authorized rebaselining. They should be read as measurement-layer evidence and as illustrations of when difficult cases must be handed off to the typed continuity rule of Section 5. Threshold crossing, recovery, or baseline shift in a figure therefore marks an operational signal, not an automatic final identity verdict.

[Original figure environments unchanged – TikZ plot for Fig 1, or Figs 2–6 with updated captions if needed; all remain as in original document.]

B Reproducibility Scope and Package Status

The complete HonestAGI-style intrinsic fingerprinting test harness (including 'anchor$_injection.py$', '$synthetic_te$ $clickverificationwithWilcoxonandbootstraptests)isavailableinthe$`fingerprint-anchor/`$directoryoftherepo$

The simulation source code and generated data for Stage-I experiments are available at the same repository.

Stage-II fingerprint verification scripts will be released upon acceptance.

Whether the same invariant can also stabilize persistent agents in extreme environments (orbital compute, Starship-scale autonomy) remains an exciting open frontier.

C Acknowledgments

The author thanks anonymous peer reviewers for detailed feedback that materially strengthened the manuscript prior to arXiv submission.

References

[1] David Wiggins. *Sameness and Substance Renewed*. Cambridge University Press, 2001.

[2] Do-hyeon Yoon et al. (HonestAGI). *Intrinsic Fingerprint of LLMs: Continue Training is NOT All You Need to Steal A Model!*. arXiv:2507.03014, July 2025.

[3] Harold Noonan and Ben Curtis. *Identity*. The Stanford Encyclopedia of Philosophy, Metaphysics Research Lab, Stanford University. https://plato.stanford.edu/entries/identity/. Accessed March 7, 2026.

[4] Max A. Freund and Richard E. Grandy. *Sortals*. The Stanford Encyclopedia of Philosophy, Metaphysics Research Lab, Stanford University. https://plato.stanford.edu/entries/sortals/. Accessed March 7, 2026.

[5] Elham Tabassi. *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. NIST Trustworthy and Responsible AI, NIST AI 100-1, National Institute of Standards and Technology, 2023. DOI: https://doi.org/10.6028/NIST.AI.100-1.

[6] Chloe Autio, et al. *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*. NIST Trustworthy and Responsible AI, NIST AI 600-1, National Institute of Standards and Technology, 2024. DOI: https://doi.org/10.6028/NIST.AI.600-1.

[7] Murugiah Souppaya, Karen Scarfone, and Donna Dodson. *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*. NIST Special Publication 800-218, National Institute of Standards and Technology, 2022. DOI: https://doi.org/10.6028/NIST.SP.800-218.

[8] Harold Booth, et al. *Secure Software Development Practices for Generative AI and Dual-Use Foundation Models: An SSDF Community Profile*. NIST Special Publication 800-218A, National Institute of Standards and Technology, 2024. DOI: https://doi.org/10.6028/NIST.SP.800-218A.

[9] Paul Groth and Luc Moreau, editors. *PROV-OVERVIEW: An Overview of the PROV Family of Documents*. W3C Note, 30 April 2013. https://www.w3.org/TR/2013/NOTE-prov-overview-20130430/.

[10] in-toto Authors. *in-toto Specification*. Project specification repository. https://github.com/in-toto/specification. Accessed March 7, 2026.

[11] Coalition for Content Provenance and Authenticity. *Content Credentials: C2PA Technical Specification, Version 2.3*. 2025. https://spec.c2pa.org/specifications/specifications/2.3/specs/C2PA_Specification.html. Accessed March 7, 2026.

[12] Aure Ecker-Fils. *Identity Doctrine: Ontology, Axioms, Collapse, and Human Interpretation*. Zenodo Preprint, February 2026. DOI: https://doi.org/10.5281/zenodo.18648150.

[13] Jeffrey Camlin. *Consciousness in AI: Logic, Proof, and Experimental Evidence of Recursive Identity Formation*. arXiv:2505.01464, May 2025.

[14] D. Sculley et al. *Hidden Technical Debt in Machine Learning Systems*. Advances in Neural Information Processing Systems (NeurIPS), 2015.

[15] Andrei Paleyes, Raoul-Gabriel Urma, and Neil D. Lawrence. *Challenges in Deploying Machine Learning: A Survey of Case Studies*. ACM Computing Surveys, 55(6), 2022. DOI: https://doi.org/10.1145/3533378.

[16] Rishi Bommasani et al. *On the Opportunities and Risks of Foundation Models*. arXiv:2108.07258, 2021.

[17] Jie Lu et al. *Learning under Concept Drift: A Review*. IEEE Transactions on Knowledge and Data Engineering, 31(12), 2018. DOI: https://doi.org/10.1109/TKDE.2018.2876857.

[18] Stephan Rabanser, Stephan Günnemann, and Zachary Lipton. *Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift*. Advances in Neural Information Processing Systems (NeurIPS), 2019.

[19] Supply-chain Levels for Software Artifacts (SLSA) Contributors. *SLSA Specification, Version 1.0*. 2023. https://slsa.dev/spec/v1.0/.

[20] Richard Ngo, Lawrence Chan, and Sören Mindermann. *The Alignment Problem from a Deep Learning Perspective*. arXiv:2209.00626, 2022.

[21] xAI. *Colossus: Training at Gigawatt Scale*. xAI Blog, 2025.